Latency Hiding

Computer Science Department
Eastern Washington University
Yun Tian (Tony) Ph.D.

Review

- A more complex kernel, Matrix-matrix multiplication.
- Query Device Property

Query Device Properties

```
cudaDeviceProp dev_ prop;
for ( i = 0; i < dev_count; i ++)
{
   cudaGetDeviceProperties( &dev_ prop,  i );
   // decide if device has sufficient resources and capabilities
}
```

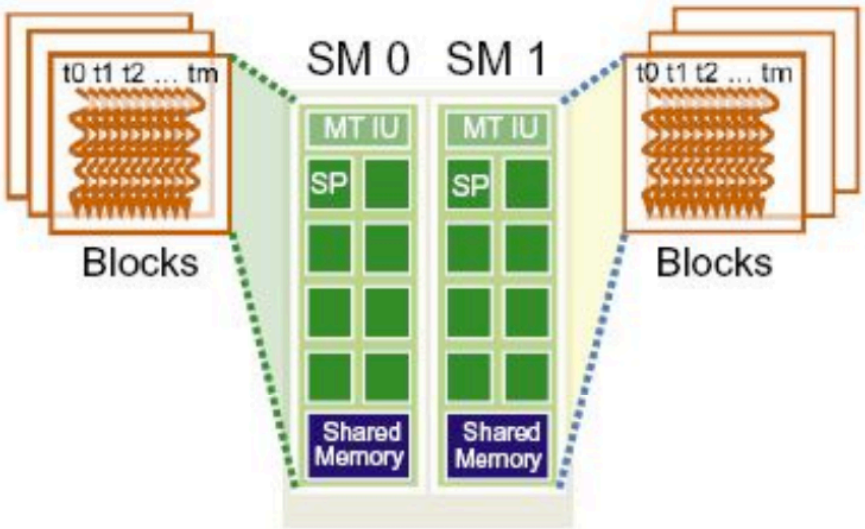
- The built-in type cudaDeviceProp is a C structure with fields that represent the properties of a CUDA device.
- Demo Code
  - /usr/local/cuda-5.5/samples/1_Utilities/deviceQuery

Outline Today

- Assign resources to Blocks
- Thread scheduling
- Latency Tolerance

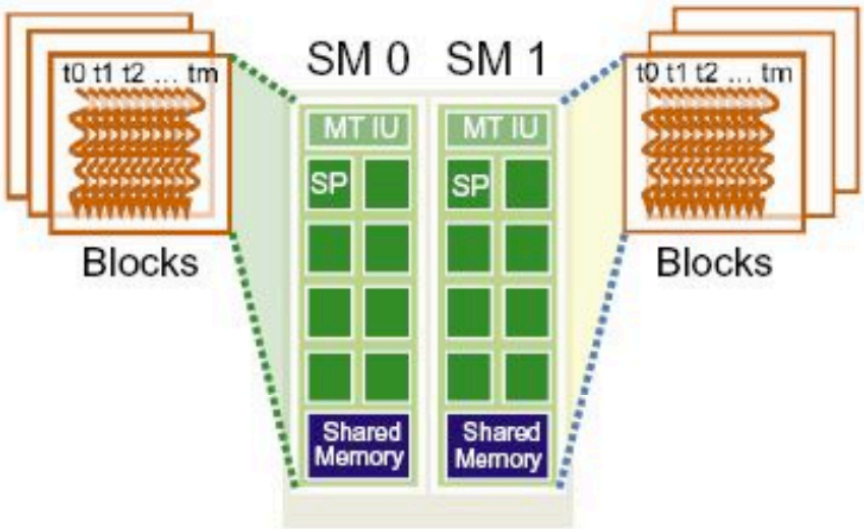
Assigning Resources to Blocks

- Threads are assigned to SMs on a block-by-block basis
- In figure right, multiple thread blocks can be assigned to each SM.



Assigning Resources to Blocks

- But there is limit on the max # of blocks to be assigned to each SM.
  - E.g. may allow up to 8 blocks that can be assigned to each SM.
    - Threads of 8 blocks actively residing in the SM.



Assigning Resources to Blocks

- Therefore, a limit on the # of blocks that can be actively executing in GPU device.
- Most grid contains many more blocks than this limit.
- Runtime sys.
  - maintains a list of blocks that need to execute,
  - assigns new blocks to SMs once they finish the blocks previously assigned to them.

Assigning Resources to Blocks

- Also, another SM resource limitation
  - Max # of threads that can be simultaneously tracked and scheduled.
  - Most recent CUDA device designs, take up to 1536 threads can be assigned to each SM.
    - Could be 6 block of 256 threads each Or 3 blocks of 512 threads each.
    - If we a device has 30 SMs, the device can have up to 46080 threads **simultaneously residing** in the device.

Thread Scheduling and Latency Hiding

- A block is divided into 32-thread units called warps.
  - # of threads in warp depends on your hardware.
  - dev_prop.warpSize returns this device property.
- Warp is the unit of thread scheduling.
- Warp assigned to a SM executes in strict SIMD
  - At any time instant, a same instruction is performed for all threads in that warp.

ing

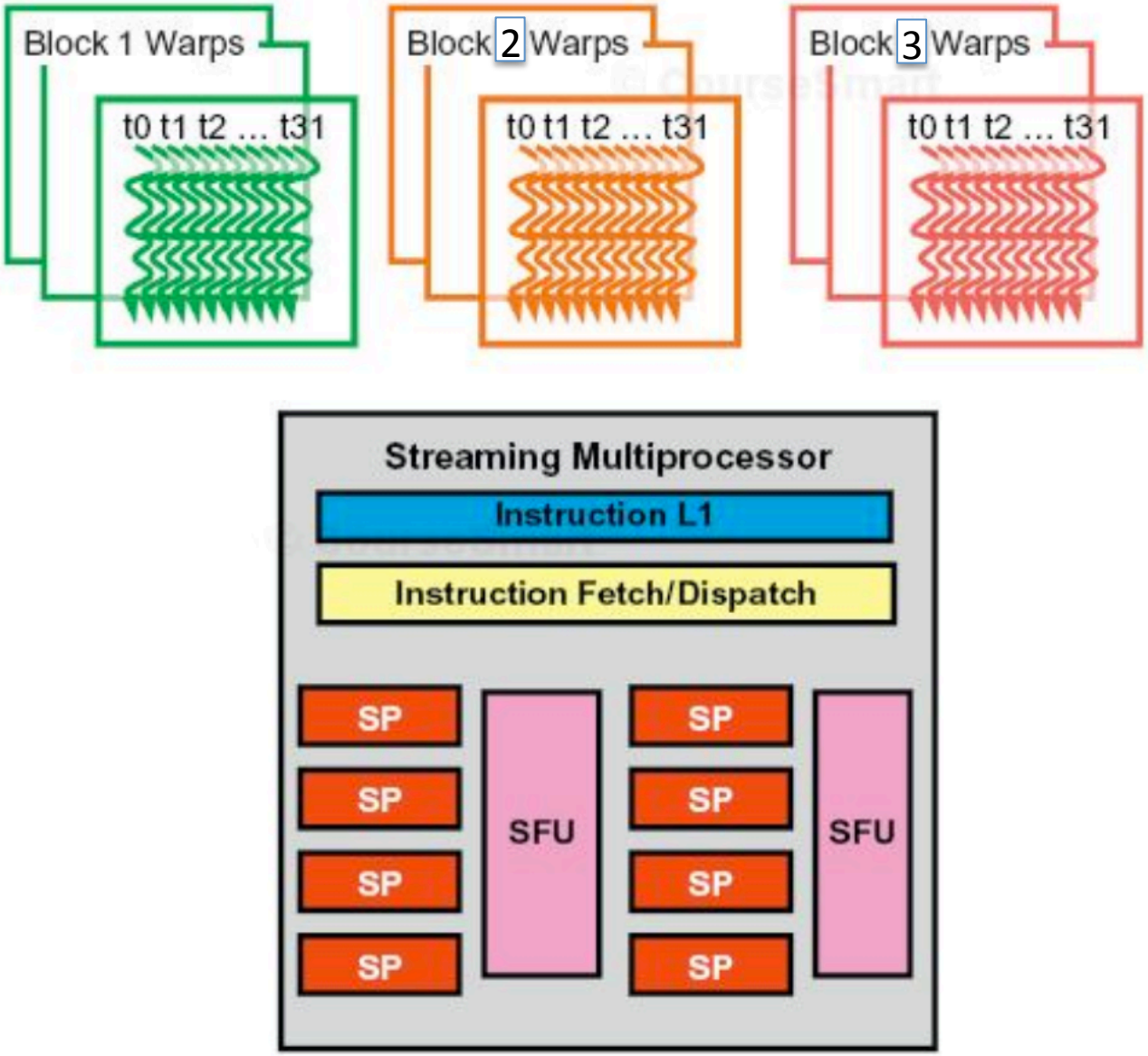


FIGURE 4.14

Blocks are partitioned into warps for thread scheduling.

Thread Scheduling and Latency Hiding

- We can calculate how many warps has been assigned to each SM.
  - With info. About # of blocks assigned to the SM
  - # of threads in each block.
- In fig 4.14, if each block has 256 threads,
  - Each block has 8 warps.
  - With three blocks in each SM,
  - We have 24 warps in each SM.

Thread Scheduling and Latency Hiding

- In general, we have fewer SPs than the # of threads assigned to each SM.
  - At any point of time, each SM executes only a small subset of threads assigned to the SM,
    - Though Newer GPUs can execute instructions for more than one Warps at a time.
- Each SM can execute a small subset of all warps at a time.

Thread Scheduling and Latency Hiding

- Why we need so many warps in an SM?
  - This is GPU design to efficiently execute long-latency operations,
    - Such as global memory access
  - If threads in a warps is waiting for the results of a previously initiated long-latency operation,
    - The warp is not selected for execution.
  - But, another resident warp that is no longer waiting for results will be selected.

Thread Scheduling and Latency Hiding

- Why we need so many warps in an SM?
  - **Latency tolerance** or **Latency hiding**
    - The mechanism of filling the latency time of operations with work from other threads.
    - Without slowing down the overall execution throughput of large number of threads residing in device.
    - In daily life in post office.
- Warp scheduling also for tolerating other type of latencies
  - E.g. branch instruction.

Thread Scheduling and Latency Hiding

- Why we need so many warps in an SM?
  - With enough warps around, at any time, hardware can find a warp to execute,
    - Make full use of the execution hardware in spite of long-latency operations.
    - With **zero-overhead thread scheduling**
      - **Selecting ready warps for execution incurs no idle time into the execution timeline.**

Thread Scheduling and Latency Hiding

- Differs from CPU design
  - CPU: branch prediction and cache
    - (reduce latency)
  - GPU: more cores and zero-overhead scheduling
    - (hide latency)

Thread Scheduling and Latency Hiding

- Little **Simplified** Exercise
- Assume a Cuda device allows up to 8 blocks and 1024 threads per SM, whichever becomes a limitation first. It allows up to 512 threads in each block.
- For matrix-matrix multiplication, should we use 8 * 8, 16 * 16, or 32 * 32 blocks?

Thread Scheduling and Latency Hiding

- Little **Simplified** Exercise
- If we use 8 * 8 blocks, → 64 threads per block, →we need 1024/64 = 12 to fully occupy an SM.
- But there is a limitation of up to 8 blocks in each SM.
- We end up with only 64 * 8 = 512 threads in each SM.
  - SMs are underutilized,
    - Fewer warps to schedule around long-latency operations.

Thread Scheduling and Latency Hiding

- Little **Simplified** Exercise
- If we use 16 * 16 blocks, → 256 threads per block, →we need 1024/256 = 4 blocks to fully occupy an SM.
  - Within the 8-block limitation.
- Good configuration
  - We have full thread capacity in each SM, 1024.
  - Maximal # of warps for scheduling around long-latency operations.

Thread Scheduling and Latency Hiding

- Little **Simplified** Exercise
- If we use 32 * 32 blocks, → 1024 threads per block, → exceeding the limitation of 512 threads per block.
  - Wrong configuration
- This is a **oversimplified** example, in chapter 5 we will consider more factors, like # of registers and shared mem, when choosing blocks dimensions.

Summary

- Assign resources to Blocks
- Thread Scheduling and Latency Hiding

Future Classes

- Set up double pointers
- Text processing on GPU.
- Memory Model of GPU device